

LibraVerse — Project Report

Sep 29, 2026 · Janci Kudana

Abstract

Libraries still depend heavily on paper registers, cash payments and plastic membership cards, and small libraries cannot afford custom software to replace them. LibraVerse is a multi-tenant Software-as-a-Service (SaaS) web platform that any library can register on to run its complete operation digitally, with only the physical books remaining physical. Each library gets an isolated, branded workspace, connects its own Razorpay account, verifies members through digital ID proof, and manages its catalog, circulation, fines, reservations, donations and reports online. Payments, whether made online or at the counter, follow a single timer-bound flow confirmed by Razorpay webhooks and pushed in real time to both the admin and the member through Socket.io. Every member receives a 3D interactive virtual membership card, rendered with Three.js, whose QR code is scanned at the counter to issue books. The platform is built on the MERN stack (MongoDB, Express, React, Node.js), is installable on phones as a Progressive Web App, and earns revenue through Free and Pro subscription plans for libraries.

Keywords: Library management, SaaS, multi-tenancy, Razorpay, real-time payments, Socket.io, Three.js, Progressive Web App, MERN stack

Project Overview

LibraVerse is a multi-tenant web platform that any library can sign up for to run its entire operation digitally. Only the physical books stay physical. Each library gets its own branded space, collects membership fees and fines through its own Razorpay account, and issues every member a 3D interactive virtual membership card in place of a plastic one.

Project type	Academic major project, full-stack web application
Target users	College, school, public and private (paid reading-room) libraries
Core idea	Everything digital except the books
Key highlights	Multi-tenant SaaS, real-time Razorpay payments (online and at the counter), 3D virtual membership card, digital ID verification

Problem Statement

Most small and medium libraries still run on paper registers, spreadsheets or outdated desktop software, and no affordable platform lets any library sign up and go fully digital with online payments. This causes six recurring problems:

1. **Manual record-keeping.** Issues, returns and member details are written by hand, which leads to errors, lost records and slow service at the counter.

2. **No digital payments.** Membership fees and late fines are collected in cash against paper receipts. Members have no proof of payment, and owners have no transparent record of collections.
3. **Physical membership cards.** Cards get lost, damaged or duplicated, and verifying a member's identity is slow and manual.
4. **No visibility for members.** Members cannot check book availability, due dates or pending fines without visiting the library.
5. **No insights for owners.** Owners cannot easily see revenue, overdue books, popular titles or member activity.
6. **High cost of custom software.** Each library would have to build or buy its own system, which small libraries cannot afford.

Problem statement: Design and develop a multi-tenant, fully digital library management platform where any library can onboard itself, verify members digitally, manage its catalog and circulation, and collect payments in real time (online or at the counter) through its own payment gateway, with a 3D virtual membership card replacing physical cards.

Existing System vs Proposed System

Today's libraries use one of three approaches, and none of them combines self-service onboarding, digital membership and real-time payments.

Aspect	Paper registers and spreadsheets	Installed library software	LibraVerse (proposed)
Setup	None, but fully manual	Installation and a server per library	Sign up online and go live
Book issue and return	Handwritten entries	Typed or barcode-based	Two QR scans at the counter
Membership card	Plastic or paper card	Printed card	3D virtual card with QR
Member ID check	Photocopy of ID	Manual	Digital ID upload and approval
Payments	Cash, paper receipt	Mostly recorded manually	Razorpay online and at the counter, cash with audit log
Payment confirmation	None	Manual entry	Real-time success screen for admin and member
Member self-service	None	Limited or none	Search, reserve, renew, pay from phone
Reports	Counted by hand	Available	Live dashboards, PDF and Excel export
Serving many libraries	Not possible	One installation per library	Many libraries on one platform
Cost for a small library	Low money, high effort	High	Free plan, paid Pro plan

Limitations of existing systems: errors and lost records, no proof of payment, lost cards, no remote access for members, no real-time insight for owners, and high setup cost for software.

Proposed Solution

LibraVerse solves these problems as a single SaaS platform shared by many libraries, each isolated from the others. Its design rests on six decisions:

- **Multi-tenant platform.** Any library registers, sets its name, logo and branches, and gets its own isolated space. Library A never sees Library B's data.
- **Everything digital except the books.** Registration, ID verification, membership cards, issue and return, receipts, notices and reports are all digital. No paper registers or plastic cards.
- **Each library uses its own Razorpay account.** Member payments go straight to the library's own account. Keys are stored encrypted.
- **One payment pattern for online and counter payments.** Every payment is a timer-bound request. When it succeeds, the admin sees "Payment Successful" and the member sees "Membership Activated" at the same instant, like the checkout flows in Swiggy, Zepto and Blinkit.
- **3D virtual membership card.** It replaces the physical card and is used for identification and book issuing at the counter.
- **SaaS subscription for libraries.** Libraries pay the platform through Free and Pro plans, which makes the product a sustainable business.

Objectives

1. Digitize every library process except the physical books.
2. Let any library onboard itself and operate in an isolated space (multi-tenancy with multi-branch support).
3. Provide secure, real-time payments through Razorpay, online and at the counter, with countdown timers and live status on both screens.
4. Replace physical cards with a 3D rotatable virtual membership card carrying a QR code.
5. Automate due dates, fines, reminders and membership expiry.
6. Verify every member's identity digitally before activating a membership.
7. Give owners real-time dashboards and exportable reports.
8. Generate platform revenue through Free and Pro subscription plans for libraries.

Feasibility Study

The project is feasible on all four counts, because it uses free, well-documented tools and free tiers for development and hosting.

Type	Question	Assessment
------	----------	------------

Technical	Can it be built with available technology and skills?	Yes. MERN, Socket.io, Three.js and Razorpay are mature and well documented, and Razorpay test mode allows full payment testing without real money.
Economic	Is it affordable to build and run?	Yes. Development tools are free, and hosting, database and file storage have free tiers sufficient for development and demonstration.
Operational	Will libraries and members be able to use it?	Yes. It runs in any browser, installs on phones as an app, and replaces familiar paper steps with simple scans and forms.
Schedule	Can it be finished within the academic timeline?	Yes, with the phased plan in the Project Timeline section and modules built in priority order.

User Roles

Four roles, each with its own permissions; the first sits above all libraries, the other three belong to one library.

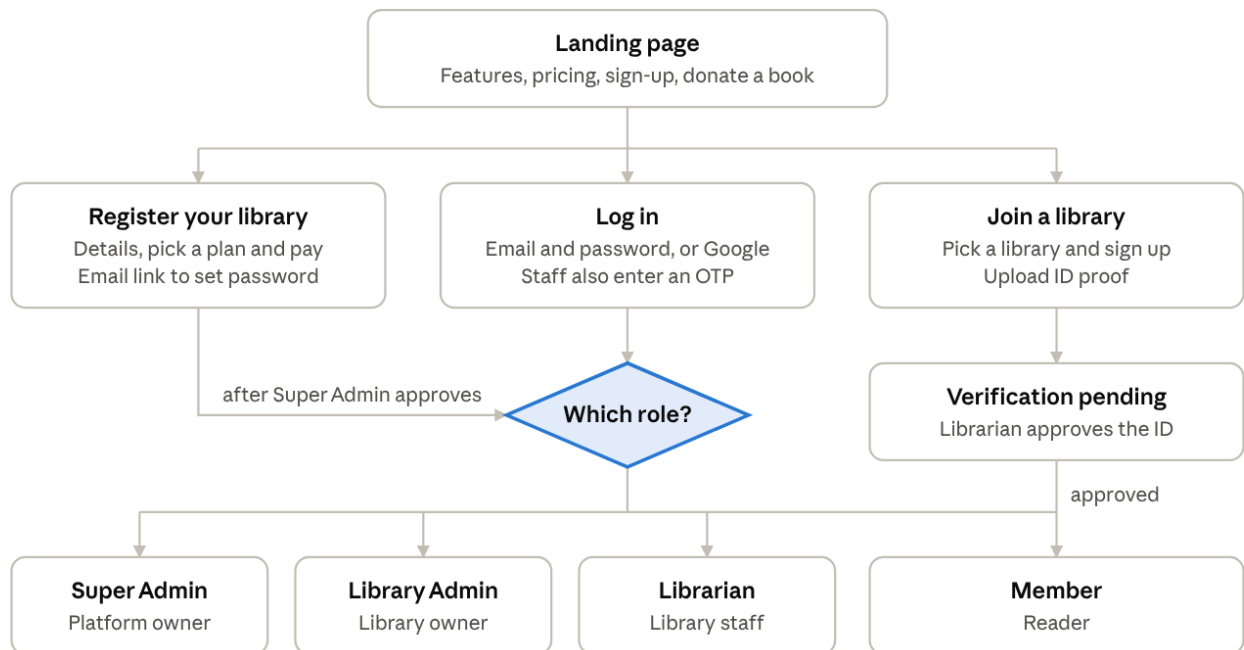
Role	Who	Main responsibilities
Super Admin	Platform owner	Approve libraries, manage Free/Pro plans, platform-wide analytics
Library Admin	Library owner	Library settings and branding, branches, staff, Razorpay keys, membership plans, reports
Librarian	Library staff	Catalog, issue and return, counter payments, ID verification, QR scanning
Member	Reader	Search, borrow, reserve, pay online, view virtual card, receive notifications

Website Flow by Role

Everyone uses one login page; after login, each role lands on its own dashboard with a sidebar menu of the pages it is allowed to open.

Login and sign-up

One login page routes each person to their own dashboard

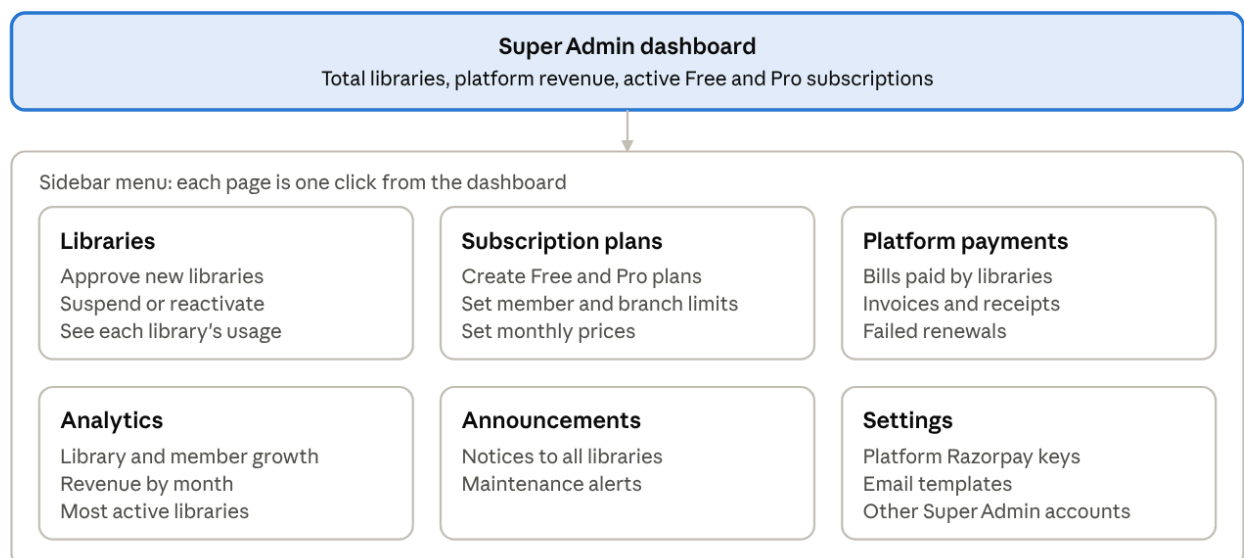


login and sign-up flow · 4 dashboards

A new member cannot log in to the member area until a librarian approves their ID; until then they see a "Verification pending" screen.

Super Admin (platform owner)

Super Admin runs the platform, not any single library

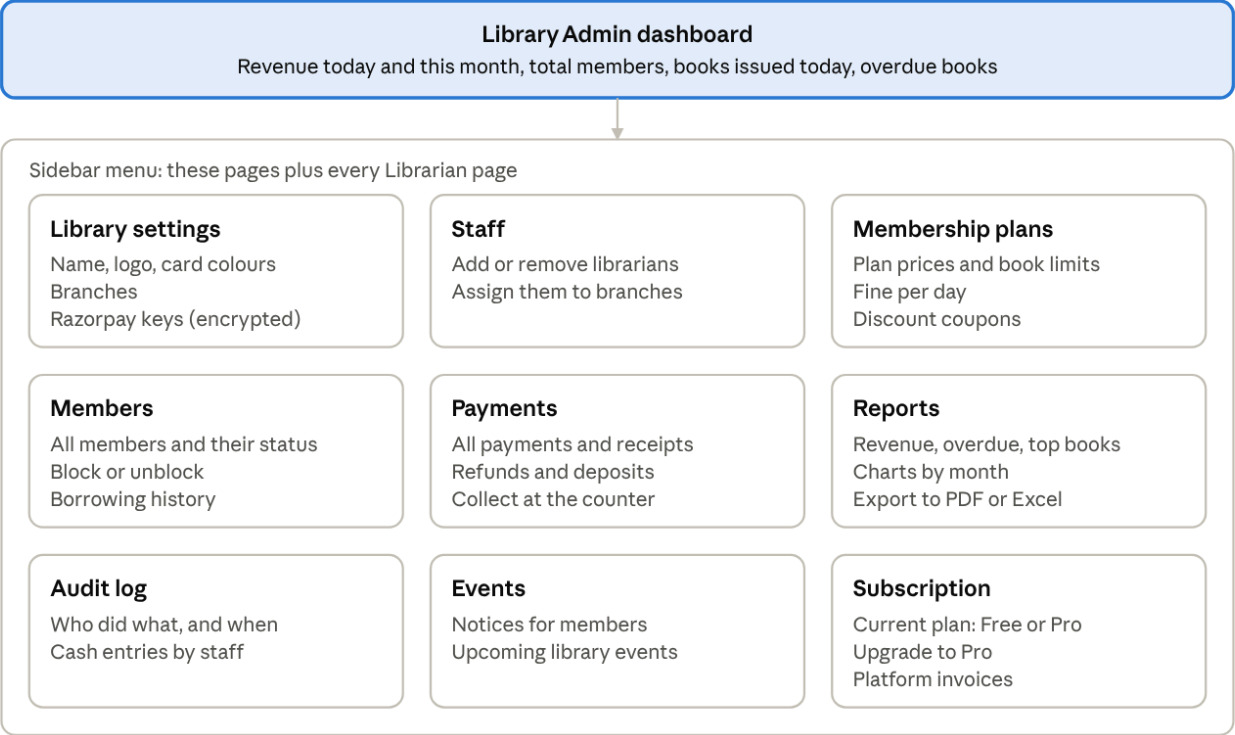


Super Admin screens · dashboard + 6 pages

The Super Admin never sees a library's books or members; only library-level totals and subscription billing.

Library Admin (library owner)

Library Admin controls one library's settings, money and staff

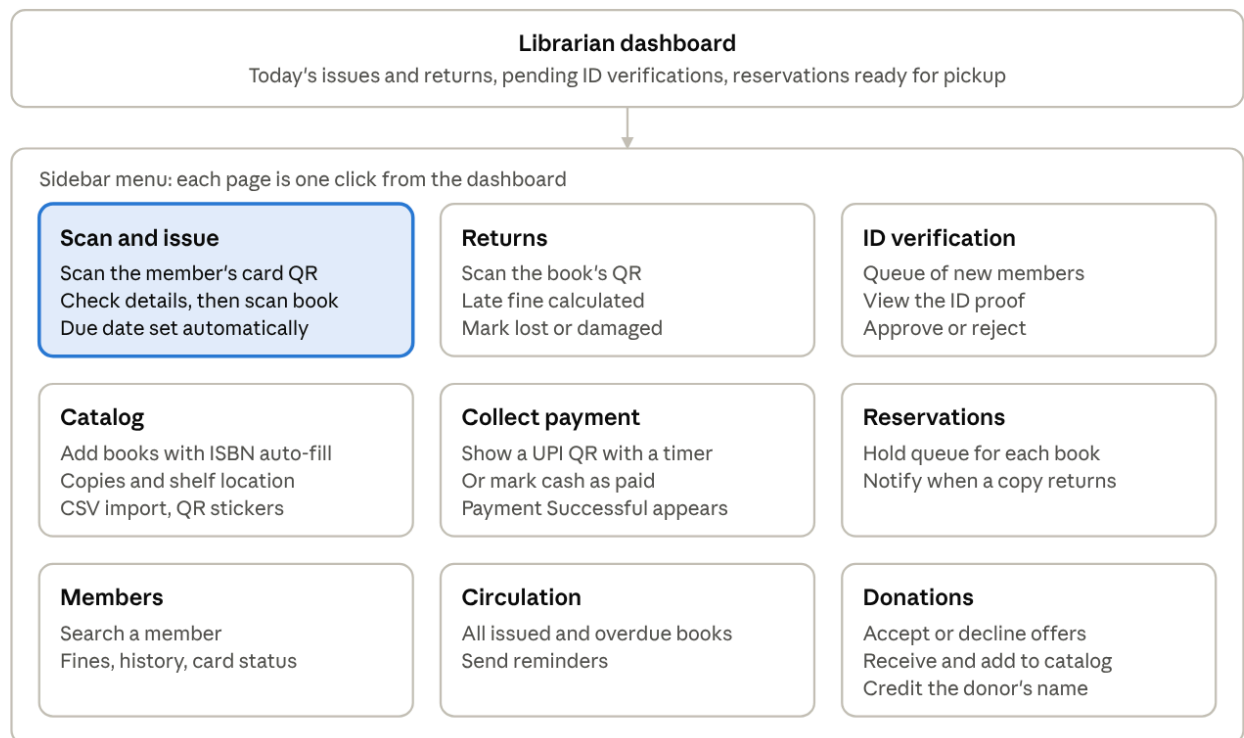


Library Admin screens · dashboard + 9 pages

The Library Admin sees only their own library, and can also do everything a Librarian does.

Librarian (library staff)

The librarian's day runs on two scans: the card, then the book

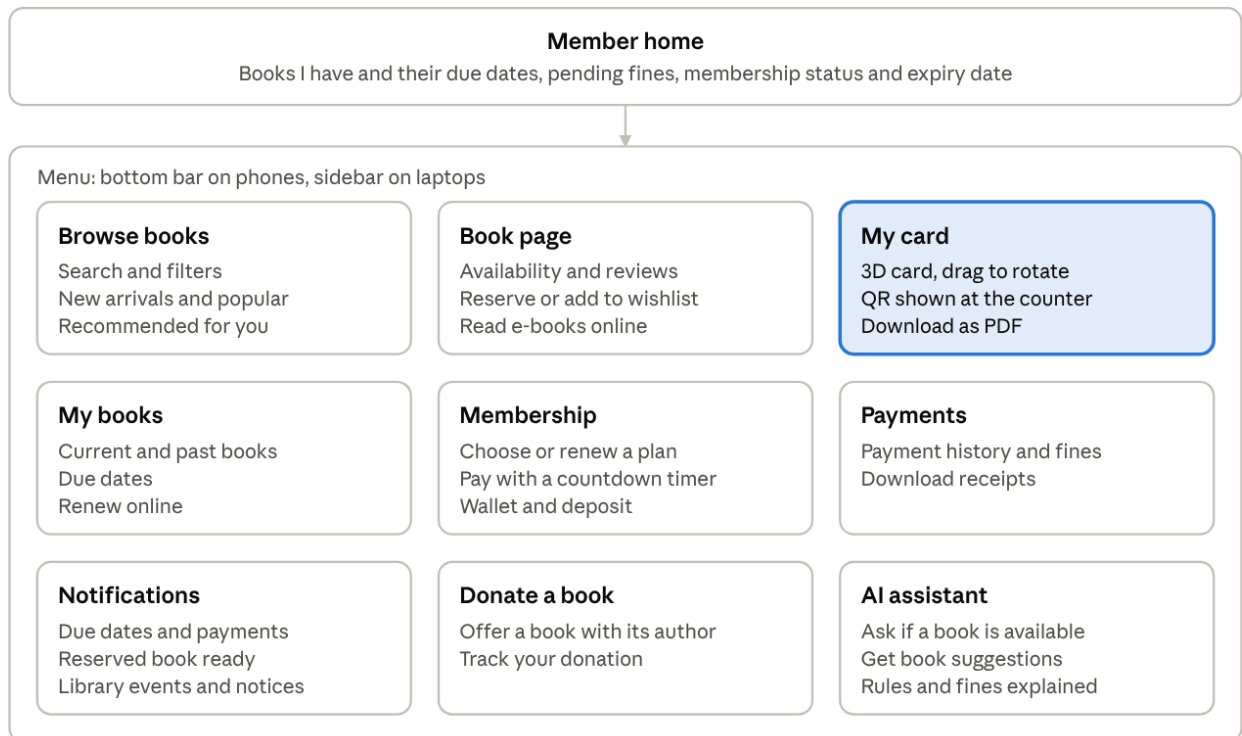


Librarian screens · dashboard + 8 pages

Issuing a book takes two scans: the member's card QR shows their details and any pending fine, then the book's QR issues that copy.

Member (reader)

Members manage borrowing, payments and their card from one home screen



Member screens · home + 9 pages

The first time a membership is activated, My card plays the reveal animation; after that it opens straight into rotate mode.

Functional Requirements

The system shall provide the following functions, grouped by the role that uses them.

ID	Role	The system shall allow...
FR-01	Visitor	a library owner to register a library, choose a plan and pay, and receive an email link to set a password
FR-02	Visitor	a person to sign up as a member of a chosen library and upload ID proof
FR-03	Visitor	anyone to offer a book donation with title, author and condition
FR-04	All users	login with email and password or Google, and password reset by OTP
FR-05	Super Admin	approving, rejecting, suspending and reactivating libraries

FR-06	Super Admin	creating and editing subscription plans and their limits
FR-07	Super Admin	viewing platform revenue, subscriptions and growth analytics
FR-08	Library Admin	setting library branding, branches and Razorpay keys
FR-09	Library Admin	adding and removing librarians
FR-10	Library Admin	defining membership plans, fine per day and coupons
FR-11	Library Admin	viewing payments, issuing refunds, and viewing reports and the audit log
FR-12	Library Admin	upgrading or downgrading the library's subscription
FR-13	Librarian	approving or rejecting member ID proofs
FR-14	Librarian	adding books and copies, with ISBN auto-fill and bulk CSV import
FR-15	Librarian	issuing a book by scanning the member's card QR and the book's QR
FR-16	Librarian	returning a book, with automatic late-fine calculation
FR-17	Librarian	creating a counter payment request with a UPI QR and timer, or recording cash
FR-18	Librarian	managing reservations and accepting and cataloguing donations
FR-19	Member	searching and filtering books, and reserving or wishlisting them
FR-20	Member	buying or renewing a membership online with a countdown timer
FR-21	Member	viewing and rotating the 3D membership card, and downloading it
FR-22	Member	viewing borrowed books and due dates, and renewing online
FR-23	Member	paying fines and downloading receipts
FR-24	Member	receiving email, SMS and push notifications

FR-25	Member	asking the AI assistant about books, rules and fines
FR-26	System	confirming payments only through verified Razorpay webhooks
FR-27	System	pushing payment results to admin and member screens in real time
FR-28	System	sending due-date, overdue and expiry reminders automatically

Non-functional Requirements

Category	Requirement
Security	Passwords hashed with bcrypt; JWT access and refresh tokens; OTP two-factor login for staff; Razorpay keys encrypted at rest; webhook signatures verified; card QR codes signed so they cannot be forged; HTTPS everywhere
Data isolation	Every record carries its library ID, and every query is filtered by it, so no library can read another's data
Privacy	ID proofs visible only to that library's staff; Super Admin sees only library-level totals
Performance	Pages load in under 3 seconds on a normal 4G connection; payment status reaches both screens within 2 seconds of the webhook
Availability	Hosted on cloud services, with the member area usable as long as the server is reachable
Usability	Responsive on phone, tablet and laptop; installable as an app; common actions (issue, return, pay) in three steps or fewer
Scalability	Stateless API servers and a cloud database, so more libraries can be added without changing the design
Auditability	Every payment, cash entry, approval and issue is recorded with who, what and when, and cannot be edited
Maintainability	Modular code by feature, environment-based configuration, and version control with Git

Hardware and Software Requirements

For development

Item	Requirement
Processor	Intel Core i5 / AMD Ryzen 5 / Apple M1 or better
RAM	8 GB minimum, 16 GB recommended
Storage	10 GB free space

Operating system	Windows 10 or later, macOS, or Linux
Software	Node.js (LTS), MongoDB Atlas or local MongoDB, Git, VS Code, Postman, a modern browser
Accounts	Razorpay (test mode), Cloudinary, MongoDB Atlas, an email service, Vercel and Render

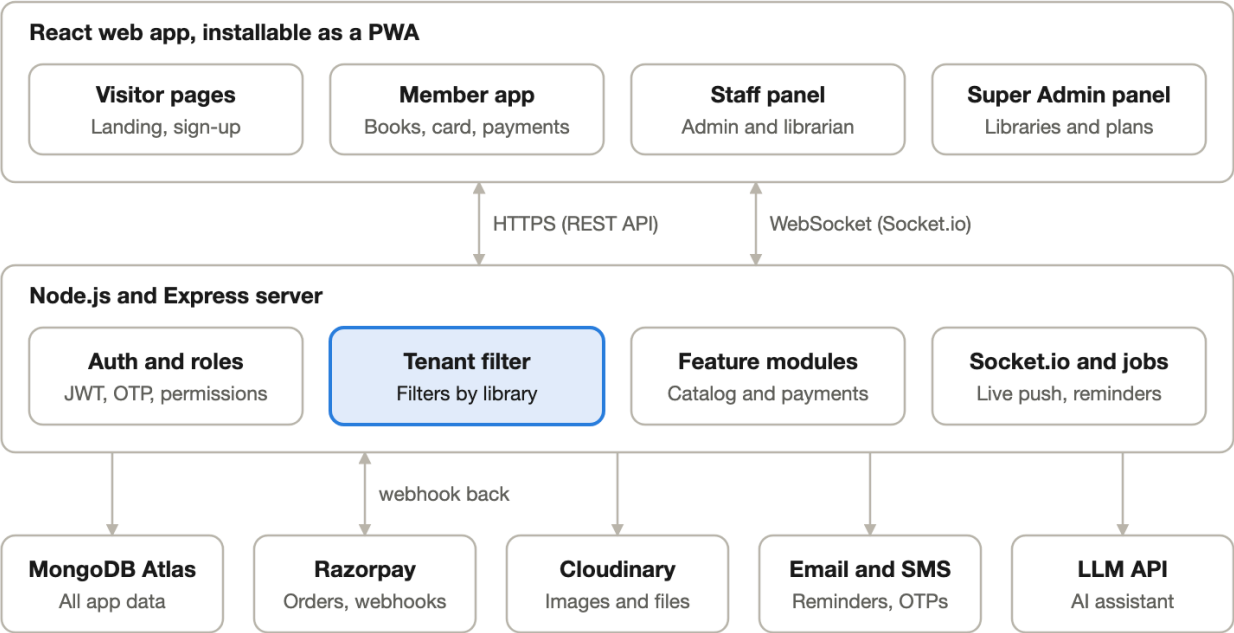
For end users

User	Requirement
Library staff	Any laptop, tablet or phone with a modern browser and a camera (webcam or phone camera) for QR scanning; internet connection
Members	Any smartphone or computer with a modern browser; a UPI app or card for online payments
Printer (optional)	Any ordinary printer to print book QR stickers

System Architecture

LibraVerse follows a three-tier architecture: a React client, a Node.js and Express API server, and cloud data and services.

One API serves every library and filters each request by library ID



system architecture · 3 tiers, 5 external services

The client talks to the server over HTTPS for normal requests and keeps a Socket.io connection open for live updates such as payment results. Razorpay calls the server back through a signed webhook when a payment finishes.

Modules

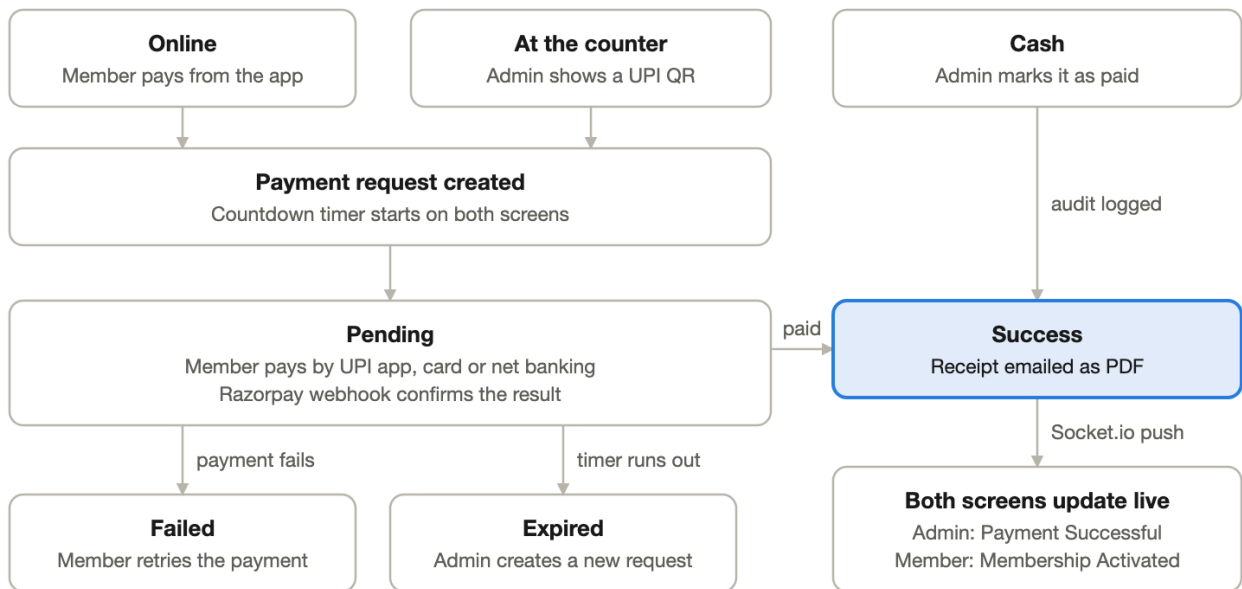
The system is divided into 14 functional modules.

#	Module	Key features
1	Platform and tenant management	Library onboarding (register, pick a plan and pay, receive an email link to set the password, Super Admin approves, library goes live), branding (logo, name, colours), multiple branches, data isolation
2	SaaS subscription billing	Free and Pro plans, plan limits (members, branches), billing through the platform's Razorpay account, upgrade and downgrade, invoices, refund if a library is rejected
3	Authentication and access control	Email/password and Google login, OTP password reset, two-factor authentication for staff, role-based permissions
4	Member onboarding and digital ID verification	Registration, ID proof upload, librarian approval or rejection, digital acceptance of terms, profile
5	Catalog management	Books and individual copies with status, categories, ISBN auto-fill, cover images, QR code per copy, shelf location, bulk CSV import, e-books and PDFs
6	Search and discovery	Search and filters, new arrivals, popular books, recommendations, ratings and reviews, wishlist
7	Circulation	QR-based issue and return (librarian scans the member's card QR, then the book's QR), borrowing limits by plan, renewals, reservation queue, self-checkout, lost and damaged handling, borrowing history
8	Payments (Razorpay)	Membership plans, late fines, lost and damaged charges, security deposit, wallet, refunds, auto-renewal, coupons, cash entries, PDF invoices and receipts
9	3D virtual membership card	First-time reveal animation, drag and tilt to rotate, tier finishes, QR code for counter identification, downloadable
10	Notifications	Email, SMS/WhatsApp, in-app and phone push alerts for due dates, overdue books, payments, reserved-book availability, membership expiry
11	Dashboards and reports	Admin and member dashboards, revenue, overdue and usage charts, PDF and Excel export, audit log
12	Events and announcements	Library notice board and event listings
13	AI library assistant	Chatbot that answers availability questions, suggests books and explains rules and fines
14	Book donations	Anyone (outside visitor or member) offers a book through a form with title, author and condition; librarian accepts or declines, receives the book and adds it to the catalog; book page shows "Donated by" (or anonymous); donor gets status emails; members earn a Contributor badge

Payment Flows

Online and counter payments share one timer-bound flow through Razorpay; cash skips the gateway but is audit-logged. The server trusts only Razorpay's verified webhook, never the browser, to mark a payment successful.

Every payment path ends in success, failure or expiry



payment lifecycle · 3 entry paths, 3 end states

When a payment succeeds, the admin's screen shows "Payment Successful" and the member's screen shows "Membership Activated" with the card reveal, both pushed live over WebSockets. A payment that fails can be retried; one that runs out of time must be requested again.

3D Virtual Membership Card

The card is modelled as a real 3D object with metallic finish and visible thickness, styled like a premium bank card, and it is the member's only membership card.

Side	Contents
Front	Library logo and name, metallic chip, a giant monogram of the library's first letter (unique per library), tier badge (Member / Gold / Elite)
Back	Embossed 16-digit membership number, Valid Thru date, Member ID, QR code scanned at the counter for issuing books, member's verified photo, library crest, signature strip

Tier finishes: Gold = champagne gold, Premium = crimson red, Elite = black metal.

Reveal animation (plays once, when the membership is first activated):

1. The screen fades to black and a crimson glow rises from the bottom.
2. The card shoots up from below, spinning fast with motion blur, and flips across several axes to show front, back and front.
3. It slows smoothly and drifts in a gentle vertical loop.
4. It settles at a tilted angle while a light sweeps across the chip.
5. The hint "Drag to rotate" appears and the card becomes interactive: drag to spin, double-tap to flip, tilt the phone to tilt the card.

After the first reveal, the card opens directly in interactive mode. Its status shows as Active, Expired or Blocked.

Database Design

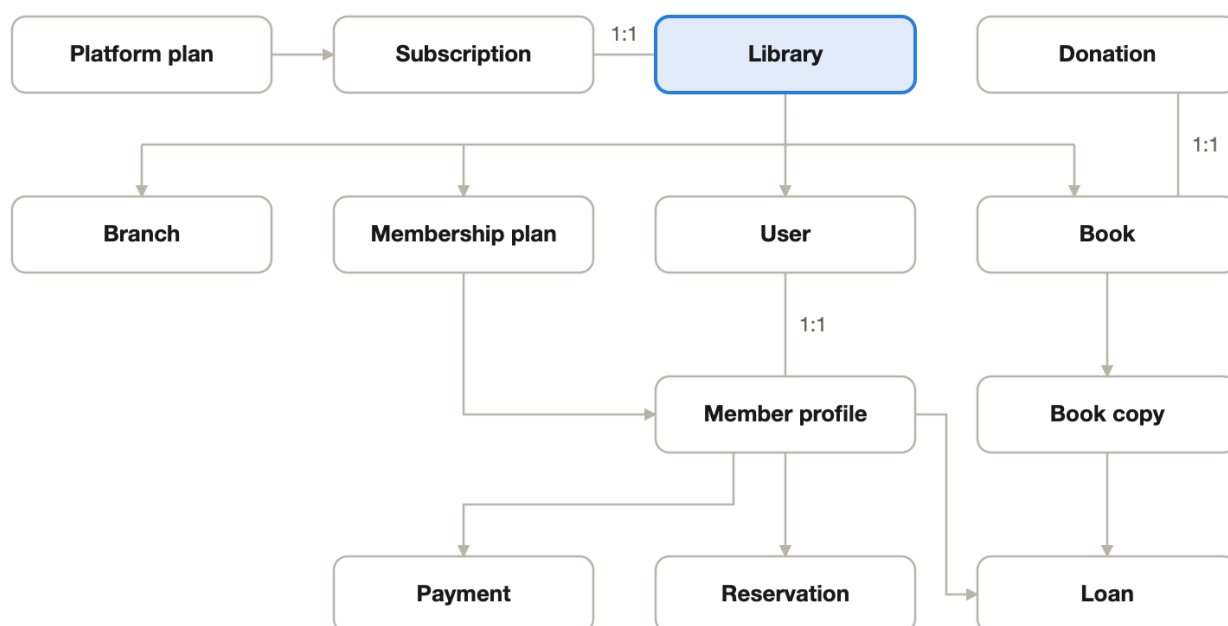
The database is MongoDB with 18 collections. Every collection except the platform-level ones carries a libraryId, which keeps each library's data separate.

Collection	Purpose	Key fields
platformPlans	Free and Pro plans sold to libraries	name, monthlyPrice, memberLimit, branchLimit
libraries	One record per library (tenant)	name, slug, logoUrl, cardColours, status (pending / active / suspended), razorpayKeyId, razorpaySecret (encrypted)
subscriptions	A library's platform plan	libraryId, platformPlanId, status, currentPeriodEnd, razorpaySubscriptionId
branches	Library locations	libraryId, name, address
users	Every login account	libraryId (empty for Super Admin), name, email, passwordHash, role, branchId, twoFactorEnabled, status
memberProfiles	Member-only details	userId, libraryId, idProofUrl, verificationStatus, membershipNo, planId, validTill, cardTier, walletBalance, badges
membershipPlans	Plans a library sells to members	libraryId, name, price, durationDays, bookLimit, finePerDay, tier
coupons	Discount codes	libraryId, code, discountPercent, validTill
books	A title in the catalog	libraryId, title, authors, isbn, category, language, coverUrl, ebookUrl, donationId
bookCopies	Each physical copy	libraryId, bookId, branchId, qrCode, shelf, status (available / issued / reserved / lost / damaged)
loans	A book issued to a member	libraryId, copyId, memberId, issuedBy, issuedAt, dueAt, returnedAt, renewals, fineAmount
reservations	Hold queue	libraryId, bookId, memberId, position, status

payments	Every payment attempt	libraryId, memberId, purpose (membership / fine / deposit / damage), amount, method (online / counter UPI / cash), razorpayOrderId, razorpayPaymentId, status (created / pending / success / failed / expired), expiresAt, receiptUrl, collectedBy
donations	Book donation offers	libraryId, donorName, donorEmail, memberId (optional), title, author, condition, status, bookId
reviews	Ratings and reviews	libraryId, bookId, memberId, rating, text
events	Notices and events	libraryId, title, date, description
notifications	Messages to users	libraryId, userId, channel, type, message, read
auditLogs	Unchangeable action history	libraryId, actorId, action, target, details, createdAt

Every record hangs off a library, which keeps tenants apart

Arrows point from one to many; one-to-one links are marked 1:1



ER diagram · 13 core collections

A loan joins one member profile to one book copy; reviews, events, notifications, coupons and audit logs also belong to a library and are left out of the diagram for clarity.

Scope

The project delivers a deployed, responsive web platform covering all 14 modules, with payments running in Razorpay test mode.

In scope

- Responsive web application for desktop, tablet and mobile browsers, installable on phones as a Progressive Web App (home-screen icon, full-screen mode, push notifications, phone tilt for the 3D card)
- Multi-tenant architecture with multi-branch support per library
- All 14 modules listed above
- Razorpay in test mode for a realistic end-to-end payment simulation; moving to live mode only requires live keys
- Real-time updates between admin and member screens using WebSockets
- Cloud deployment with a live demo URL

Out of scope

- Separate native Android and iOS apps (the installable web app covers mobile use instead)
- Book purchasing and vendor management
- Study room and seat booking
- Entrance check-in and physical hardware such as RFID gates and barcode printers (QR scanning by phone or webcam happens only at the counter)
- Real-money transactions during the demonstration
- Physical delivery of books

Technology Stack

The platform uses the MERN stack, extended with Socket.io for real-time updates and Three.js for the 3D card.

Layer	Technology
Frontend	React (Vite), Tailwind CSS, Framer Motion, vite-plugin-pwa (installable app, offline shell, Web Push)
3D card	Three.js with React Three Fiber and Drei
Backend	Node.js, Express
Database	MongoDB with Mongoose; every record tagged with its library ID for isolation
Real-time	Socket.io
Payments	Razorpay: Orders, Payment Links, QR Codes, Webhooks, Subscriptions
Authentication	JWT with refresh tokens, Google OAuth, bcrypt
File storage	Cloudinary (book covers, ID proofs, e-books)
Notifications	Nodemailer or Resend (email), Twilio or MSG91 (SMS and WhatsApp)
PDF and QR	PDFKit, qrcode, html5-qrcode scanner
AI assistant	LLM API (for example Claude)
Deployment	Vercel (frontend), Render or Railway (backend), MongoDB Atlas

Testing Plan

The system will be tested at four levels, with payments tested end to end in Razorpay test mode.

Level	What is tested	Tools
Unit testing	Fine calculation, borrowing limits, payment state changes, permission checks	Jest
API testing	Every endpoint, including wrong roles and another library's data	Supertest, Postman
Integration testing	Razorpay orders and webhooks, Socket.io events, email delivery	Razorpay test mode, webhook tunnel
User acceptance testing	Full journeys by each role on phone and laptop	Manual test scripts

Key test cases

ID	Scenario	Expected result
TC-01	Member pays for a plan online and the payment succeeds	Admin sees Payment Successful, member sees Membership Activated and the card reveal, receipt emailed
TC-02	Member lets the payment timer run out	Payment marked Expired; no membership activated
TC-03	A fake success message is sent without a valid webhook signature	Rejected; payment stays Pending
TC-04	Librarian records a cash payment	Membership activated; receipt generated; audit log entry with the librarian's name
TC-05	Librarian scans a card for a member with a pending fine	Issue blocked with the reason shown
TC-06	Member returns a book 3 days late at ₹5 per day	Fine of ₹15 added to the member's account
TC-07	A Library A user requests Library B's data	Access denied
TC-08	Member with no approved ID tries to buy a plan	Blocked with "Verification pending"
TC-09	A reserved book is returned	Next member in the queue notified
TC-10	Super Admin rejects a library that paid for Pro	Library stays inactive; refund issued
TC-11	Visitor donates a book and the librarian accepts it	Book added to the catalog with "Donated by" credit

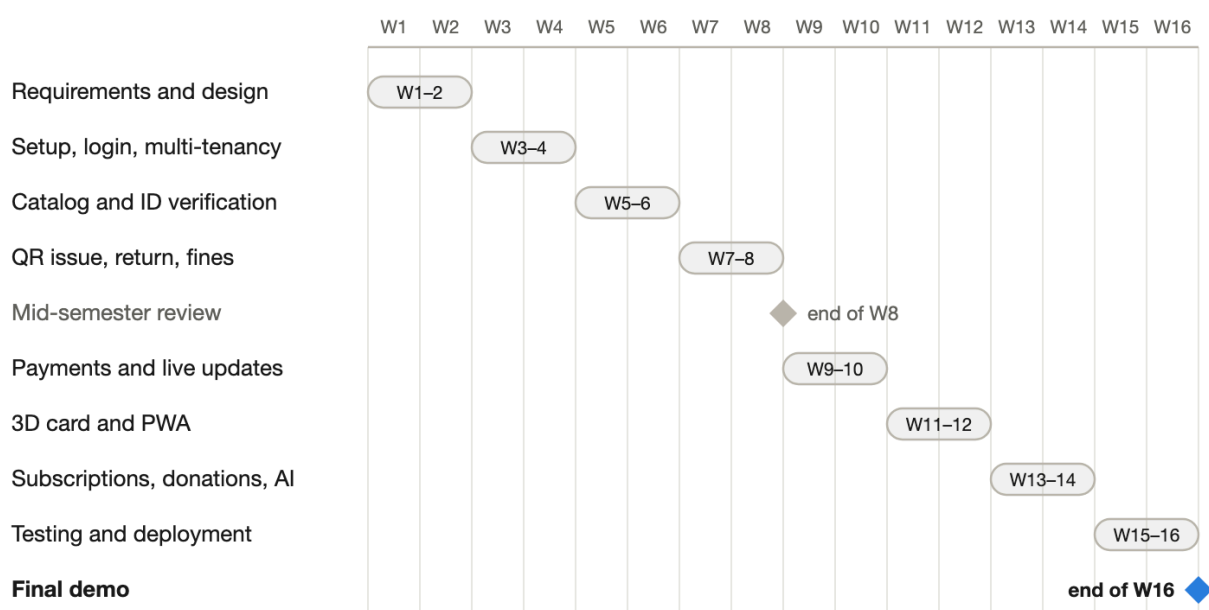
TC-12	App installed on a phone from the browser	Opens full screen with its own icon; push notifications received
-------	---	--

Project Timeline

The work is planned over a 16-week semester in eight two-week phases, building the core library features before payments and the 3D card.

Payments start after the mid-semester review; testing takes the last two weeks

16-week semester plan; each column is one week



project timeline · 8 phases over 16 weeks

By the mid-semester review, login, multi-tenancy, catalog and circulation are working; the second half adds payments, the 3D card, the remaining modules and testing. Week numbers can be mapped to your semester's actual dates.

Expected Outcomes

For libraries and members

- A library goes from sign-up to fully operational without any paperwork.
- Every payment, online or at the counter, is traceable, receipted and visible to both parties in real time.
- Members manage borrowing, fines and membership from their phone, and carry their card digitally.
- Owners see revenue, overdue books and usage at a glance.

Technical learning outcomes

- Multi-tenant SaaS architecture with data isolation and subscription billing
- Payment gateway integration with webhooks, signature verification and payment state management
- Real-time communication with WebSockets
- 3D graphics and animation on the web with Three.js
- Secure authentication, role-based access control and encrypted storage of third-party keys
- Cloud deployment of a full-stack application

Future Scope

- Native Android and iOS apps built on the same backend
- Book purchasing and vendor management for libraries
- Study room and seat booking with paid slots
- RFID gates and self-service kiosks for fully automatic entry and checkout
- Inter-library loans between libraries on the platform
- Regional language support for the interface
- Recommendations improved with machine learning on borrowing history
- Split payments through Razorpay Route so the platform can take a commission automatically

Conclusion

LibraVerse replaces paper registers, cash receipts and plastic cards with a single digital platform that any library can adopt without buying or installing software. Its real-time payment flow gives both the library and the member instant, verifiable proof of every payment, whether made online or at the counter. The 3D virtual card, digital ID verification and QR-based issuing make daily library work faster and more secure. As a multi-tenant SaaS with subscription billing, the project demonstrates not only full-stack development skills but also the design of a product that can serve many libraries as a real business.

References

1. [React documentation](#)
2. [Node.js documentation](#)
3. [Express documentation](#)
4. [MongoDB documentation](#)
5. [Mongoose documentation](#)
6. [Socket.IO documentation](#)
7. [Three.js documentation](#)
8. [React Three Fiber documentation](#)
9. [Razorpay documentation](#)
10. [Progressive Web Apps on web.dev](#)
11. [Cloudinary documentation](#)
12. [OWASP Top Ten web security risks](#)